

```
%macro io 4
    mov rax,%1 ; System call number
    mov rdi,%2 ; File descriptor (1 for stdout, 0 for stdin)
    mov rsi,%3 ; Buffer (data to write or read)
    mov rdx,%4 ; Length of buffer
    syscall ; Invoke system call
%endmacro
```

```
%macro exit 0
    mov rax,60 ; Exit system call
    mov rdi,0 ; Exit code 0 (success)
    syscall ; Invoke system call
%endmacro
```

```
section .data
    msg1 db "Write an x86/64 ALP to find the largest of given 5 64-bit hexadecimal numbers from user.",10,
        "Name:- Sumaanyu",10,"Roll:- 7256",10,"Date of Performance:- 28/02/2025",10
    msg1len equ $-msg1 ; Length of msg1
    msg2 db "Enter 5 64bit hexadecimal numbers (0-9,A-F only): ", 10
    msg2len equ $-msg2 ; Length of msg2
    msg3 db "5 64bit hexadecimal numbers are: ", 10
    msg3len equ $-msg3 ; Length of msg3
    newline db 10 ; Newline character
    msg4 db "The largest number is: "
    msg4len equ $-msg4 ; Length of msg4
    errMsg db "You entered a wrong hexadecimal number"
    errLen equ $-errMsg ; Length of errMsg
```

```
section .bss
    asciinum resb 17 ; Buffer to store user input
    hexnum resq 5 ; Array to store 5 hexadecimal numbers
```

```
section .text
global _start
_start:
    io 1,1,msg1,msg1len ; Print msg1 (problem statement and details)
    io 1,1,msg2,msg2len ; Print msg2 (prompt user for input)

    mov rcx,5 ; Loop counter for 5 inputs
    mov rsi,hexnum ; Pointer to store numbers in hexnum array

next1:
    push rsi ; Save rsi
    push rcx ; Save rcx
    io 0,0,asciinum,17 ; Read 16-character hexadecimal input
    call ascii_hex64 ; Convert ASCII to hex
    pop rcx ; Restore rcx
    pop rsi ; Restore rsi
    mov [rsi],rbx ; Store converted hex value in array
    add rsi,8 ; Move to next storage location
    loop next1 ; Repeat for all 5 numbers
```

```

io 1,1,msg3,msg3len ; Print msg3 (showing stored numbers)
mov rsi,hexnum      ; Reset rsi to start of hexnum array
mov rcx,5           ; Loop counter for 5 numbers

```

```

next2:
    mov rbx,[rsi] ; Load current hex number
    push rsi      ; Save rsi
    push rcx      ; Save rcx
    call hex_asci64 ; Convert hex to ASCII and print
    pop rcx       ; Restore rcx
    pop rsi       ; Restore rsi
    add rsi,8     ; Move to next number
    loop next2    ; Repeat for all 5 numbers

```

```

; Find the largest number
mov rsi,hexnum ; Reset rsi to start of hexnum array
mov rcx,5      ; Loop counter for 5 numbers
mov rbx,0      ; Initialize max number to 0

```

```

next5:
    mov rax,[rsi] ; Load current number
    cmp rax,rbx   ; Compare with max found so far
    jle skip1     ; If smaller or equal, skip update
    mov rbx,rax   ; Update max number
skip1:
    add rsi,8     ; Move to next number
    loop next5    ; Repeat for all 5 numbers

```

```

io 1,1,msg4,msg4len ; Print msg4 (largest number message)
call hex_asci64     ; Convert and print the largest number
exit                ; Exit program

```

```

; Function to convert ASCII hex string to 64-bit integer
ascii_hex64:

```

```

    mov rsi,asciinum ; Pointer to input string
    mov rbx,0        ; Clear result register
    mov rcx,16       ; Loop counter for 16 hex digits

```

```

next3:
    rol rbx,4 ; Left shift rbx by 4 bits
    mov al,[rsi] ; Load current character

```

```

; Check for invalid hex character
    cmp al,29h ; Less than '0'
    jbe err
    cmp al,40h ; Between '9' and 'A'
    je err
    cmp al,67h ; Greater than 'f'
    jge err
    cmp al,47h ; Between 'F' and 'a'
    jge checkfurther

```

jmp operations

checkfurther:

cmp al,60h ; Between 'Z' and 'a'  
jbe err

operations:

cmp al,39h ; If between 'o' and '9'  
jbe sub30h  
cmp al,46h ; If between 'A' and 'F'  
jbe sub7h  
sub al,20h ; Adjust for lowercase letters

sub7h:

sub al,7h ; Adjust ASCII 'A-F' to hex values

sub30h:

sub al,30h ; Convert 'o-9' to hex values

jmp skip

err:

io 1,1,errMsg,errLen ; Print error message  
io 1,1,newline,1 ; Print newline  
exit

skip:

add bl,al ; Store converted hex digit in rbx  
inc rsi ; Move to next character  
loop next3 ; Repeat for all 16 characters

ret

; Function to convert 64-bit integer to ASCII hex string

hex\_ascii64:

mov rsi,asciinum ; Pointer to output string buffer  
mov rcx,16 ; Loop counter for 16 hex digits

next4:

rol rbx,4 ; Left shift rbx by 4 bits  
mov al,bl ; Extract least significant hex digit  
and al,0fh ; Mask to get last 4 bits

cmp al,9 ; Check if digit is 0-9  
jbe add30h  
add al,7h ; Adjust for 'A-F'

add30h:

add al,30h ; Convert to ASCII  
mov [rsi],al ; Store in output buffer  
inc rsi ; Move to next character  
loop next4 ; Repeat for all 16 digits

```
io 1,1,asciinum,16 ; Print ASCII hex number  
io 1,1,newline,1 ; Print newline  
ret
```